

P1767R0

Packaging C++ Modules

Published Proposal, 2019-06-16

This version:

<http://wg21.link/p1767r0>

Author:

[Richard Smith](#) (Google)

Audience:

SG15

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper describes one concrete possibility for a packaging mechanism for C++ code.

Table of Contents

- 1 Problem**
- 2 Scope**
- 3 Approach**
 - 3.1 Packages
 - 3.2 Manifests
 - 3.3 Package names and uniqueness
 - 3.4 Package names and linkage
 - 3.5 Package name conventions
- 4 Consequences**

- 4.1 For C++ library authors
- 4.2 For C++ library packagers
- 4.3 For C++ build system vendors
- 4.4 For C++ compiler vendors
- 4.5 For C++ tool vendors

Index

Terms defined by this specification

§ 1. Problem

A C++ project (a program or library) typically has dependencies on libraries provided by third parties. When using those libraries, the build system of the project needs to be aware of how to make the dependencies visible to the build of the project.

In C++17 and before, there are a number of necessary pieces for using such a dependency, typically including:

- A set of include paths
- A set of predefined compiler macros
- A set of libraries to link against

... and possibly other things too.

In C++20, this becomes more complicated, because in general we additionally need to build module interfaces and header units as part of the build of the project. Some build-system-independent mechanism for providing this information would be highly valuable.

§ 2. Scope

There are (at least) three different levels at which C++ code is distributed:

1. Source distribution (eg, I download your library from github, or I check out a project that I intend to hack on)
2. Precompiled library distribution to developers (eg, I install a package with a package manager, or I build a library that I downloaded and install its built components somewhere for later use by other code)

3. Binary-only distribution to end-users (eg, the product made available for download on some company's website)

This paper is concerned primarily with the second level. While its approach has implications for the first and third level, the intent is to not constrain the build systems and development techniques used for package maintainers nor the ways in which C++ projects are distributed and installed on end-user systems.

This paper doesn't intend to provide or describe a complete, finished solution for any of the problems it touches upon. Instead, the hope is that this provides a venue for discussion of this approach that might lead to a more concrete finalized solution.

§ 3. Approach

§ 3.1. Packages

A **package** is a collection of C++ file system artifacts that are built and installed together in a specific build configuration. A package typically contains:

- A package [manifest](#)
- Some translated translation units, in the form of prebuilt libraries (`.lib` / `.a` / `.dll` / `.so` / `.dylib` / ...),
- Some module interface units
- Some header units
- Some textual header files

(It would be possible to ship some compiler-specific module representation ("CMI") files for a specific compiler along with a package, and this is not precluded by the model of a package. However, such files should be thought of as strictly an optional extra that does not replace the necessity to provide sources for module interfaces and header units.)

Modules and packages are different levels of a hierarchy of components:

- A package contains zero or more complete libraries. (A single library cannot be split across packages.)
- A library (`.lib` / `.a` / `.dll` / `.so` / `.dylib` / ...) contains zero or more complete modules, and zero or more complete non-module translation units.
- A module contains zero or more module units.

A module is a coherent unit of encapsulation and isolation and provides an encapsulation boundary, whereas a package is a coherent bundle of artifacts that should be distributed together, and so instead provides a distribution boundary.

A package is intended to be independent of the build system that produced it and independent of the build systems that will consume it.

EXAMPLE 1

On Linux distributions, package management systems are used to install packages. On my development Debian machine, boost is divided into 30 different `-dev` packages, each of which is a package in the sense defined in this document.

§ 3.2. Manifests

A **package manifest** is a file (in a specific new file format), distinct from the C++ source code, that describes how a collection of source code is assembled to form a package. For example, a package manifest identifies where the interface files necessary to use the package can be found (both header units and module interface units), what configuration settings are necessary to correctly build BMIs from those interface files, the include paths and other flags that must be used in code that directly depends on the package, and the dependencies of the package on other packages.

The manifest describes the specific instance of the package as installed on the system. Dependencies would typically be described by a file system path to the package manifest files corresponding to those dependency packages. To this end, package manifest files may need to be distributed as "skeleton" files that are configured to contain the correct paths at installation time.

Generation of a package manifest file will in general need information from multiple sources.

EXAMPLE 2

A package manifest file for a `.deb` package on a Debian system might be generated as follows:

- The author of the source package writes a skeleton package manifest file describing properties of the package that do not depend on how it is configured or installed.
- The configure script of the package extends the skeleton manifest file to describe the concrete configured dependencies and some of the build flags.
- The `install` stage of the package installs a package manifest file alongside the contents of the package, with the paths in the manifest file describing the installed locations of the package's contents.
- The `.deb` build script tweaks the paths to describe the location of the package on the target system instead of the locations on the build machine and sets the package name to the right name for the installed package.

The intent is that SG15 will specify a concrete package manifest file format, describing exactly how the requisite information will be encoded. However, this document does not propose any concrete file format for package manifests. The author believes that a plain-text -- possibly YAML -- format, with a suitable schema to encode all currently-known necessary information, and room for extensibility, would likely be a reasonable choice.

§ 3.3. Package names and uniqueness

One important problem to solve is that of module name uniqueness. Broadly-speaking, if we wish to avoid module name collisions, we need to ensure that there is some collision-free namespace in which module names live. Other languages deal with this in various ways, such as:

- An external unique name assignment system can be used to generate names. (For example, Java uses DNS as its source of uniqueness.)
- A language can choose One True Package System and use its package names as the source of uniqueness. (For example, the Hackage package archive serves this role for Haskell.)
- Allow customization of the search path to resolve name ambiguities. (Python permits this.)

This document proposes the following approach:

- Packages have **package names** that are unique on the system on which the package resides, but not necessarily globally unique.

- The name of a package is determined by the process that builds and installs the package, and *not* by the author of the package.
- Two same-named modules provided by different packages are nonetheless different modules (and hence there are no linker-level collisions between modules with the same name in different packages).
- A package manifest can optionally specify a ***module name prefix*** for each of its dependencies. This prefix specifies how the dependency's modules will be named within the depending module. This allows two same-named modules from different packages to be imported into the same source file.

EXAMPLE 3

Suppose that a project depends on two packages, YouEye and Fizix. Configuration of the project resolves those dependencies as follows:

- YouEye's package manifest is `/usr/share/youeye/youeye.cpkg`, which says the package name is `sys:youeye-3.4`. (This package was installed by the system's package manager, so the system's package manager is responsible for picking a unique name for the package, and it picked a name based on the package manager's name for the package.)
- Fizix's package manifest is `/home/myuser/fizix/fizix.cpkg`, which says the package name is `user:fizix`. (This package was installed by the user, so the user is responsible for picking a unique name for the package. They accepted the default package name suggested by Fizix's `configure A`) script.

The project specifies a module name prefix for its YouEye dependency of `deps.youeye` and a module name prefix for its Fizix dependency of `deps.fizix`. Code in the project can then import modules from both dependencies as:

```
export module mylib;
import deps.youeye.widgets.box; // import widgets.box from sys:youeye-3.
import deps.fizix.widgets.box;  // import widgets.box from user:fizix-4.
```

The package manifest for this project would describe the dependency on the package manifests for YouEye and Fizix, along with the module import prefixes, so that a consumer of this project can compile a BMI from its module interface.

§ 3.4. Package names and linkage

Because we require that modules in different packages are different modules even if they have the same module name, we need a mechanism for the linker to tell symbols from the two modules apart. This can be accomplished using the same techniques that are used to ensure that entities from different modules that have the same name are distinguished (eg, by name mangling), or by using another technique, such as:

- On ELF targets, use of symbol versioning information
- On MachO targets, use of two-level namespacing
- On PE/COFF targets, symbols from distinct `.dll`s can have the same name without collision

Note that the allowance of module name collisions between distinct packages permits multiple versions of a library to be used within a program (as part of multiple distinct packages). This may be inadvisable in some cases (for example, a "global" registry from package `foo-1.4` would not list entities registered with package `foo-1.5`) and some way to disallow that as optional "conflict" information in the package manifest may be useful.

§ 3.5. Package name conventions

We propose the following strawman package naming convention:

- A package name is a `:`-separated sequence of name pieces, where each level before the last describes a system that guarantees the uniqueness of the name.
- Names of the form `sys:<name>` are reserved for the system's package manager. The package name will generally correspond with the name assigned by that package manager.
- Names of the form `user:<name>` are reserved for the case where the end-user is responsible for ensuring uniqueness; this should be the default for packages built from source repositories.
- Otherwise, responsibility for uniqueness of names in the top-level namespace lies with the administrator of the system in question. For example, if the system administrator installs Homebrew in `/usr/local`, they might assign it `brew:` as a prefix for packages that Homebrew installs, but if a user installs Homebrew into their home directory, it could use `user:brew-2.1.5:` as its package name prefix.

§ 4. Consequences

§ 4.1. For C++ library authors

Your build system will need to be extended to produce a package manifest file describing your package and its dependencies. Your build configuration system (`configure` script, or meta-build system, or...) will need to know how to find the package manifest files for your direct dependencies (or how to ask the user to say where they are).

You can continue to build your code however you like, so long as you use the compilation flags required by your dependencies (much like the status quo).

§ 4.2. For C++ library packagers

You will need to ensure that package manifest files exist (and may need to author such files for, say, C libraries that provide header files that should be consumed as header units) and that the package names used by them are unique.

§ 4.3. For C++ build system vendors

You will need to parse package manifest files, and figure out suitable build rules and compiler flags for a package's transitive dependencies. You will need to cause the necessary .BMI files to be built as inputs to their consumers.

You will need to produce package manifest files for libraries that you build and install, so that those libraries can be consumed by downstream builds.

§ 4.4. For C++ compiler vendors

You have options:

- You could do nothing and expect the build system to take care of everything for you and provide you with all the BMIs you need
- You could provide a module-mapper-style interface that can talk to a build system component that uses package manifest files to determine how to build BMI files on demand
- You could accept and parse package manifest files yourself, and implicitly build BMI files on demand

All of these options might make sense in different scenarios.

§ 4.5. For C++ tool vendors

You will probably want to be given the package manifest files of dependencies of any particular compilation. With those and the compilation command, you have sufficient information to parse and process the module interface units of dependencies of the current compilation without needing to understand the formats of BMI files or how they are built.

§ Index

§ Terms defined by this specification

[module name prefix](#), in §3.3

[package](#), in §3.1

[package manifest](#), in §3.2

[package name](#), in §3.3